

DATA STRUCTURES

Teacher:

刘志昊

Email: lzh@seu.edu.cn

QQ: 65710860

东南大学九龙湖校区

计算机楼

Teaching assistant:

赵思齐 2990372954@qq.com

QQ课程群:



DS2025秋-JLH

群号: 1064790438



Main Textbook:

*Fundamentals of Data Structure in C++,
2E, Silicon Press 2007*

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta

References:

1. Ellis Horowitz, Sartaj Sahni, Dinesh Mehta, Fundamentals of Data Structure in C++, Silicon Press, 1995
2. 殷人昆, 数据结构(用面向对象方法与C++语言描述, 第2版), 清华大学出版, 2007
3. 金远平, 数据结构(C++描述), 清华大学出版社, 2005

4. Ellis Horowitz, StrtajSahni, Dinesh Mehta, Fundamentals of Data Structure in C, Silicon Press, 1995
5. Ellis Horowitz, StrtajSahni, Dinesh Mehta, 数据结构基础(C语言版,第2版),朱仲涛译,清华大学出版社,2009
6. Bradley N. Miller, David L. Ranum, Python 数据结构与算法分析(第2版),吕能,刁寿钧译,人民邮电出版社,2019
- 7.王伟,姜浩,数据结构基础实验指导,东南大学出版社, 2019

Evaluation:

- **Course Attendance, Discussion and Exercises: 20%**
- **Experiments: 30%**
- **Final Examination (Textbook and Course Notes allowed): 50%**

About Exercises and Experiments

- Exercise and Experiment questions will be listed on **the last page of the slides** in each chapter.
- You **must** complete **at least half** of all the questions. Two students can **cooperate** to complete all the questions.
- You should submit your answers **every two weeks** to the teaching assistant **by email**.
- No plagiarism!

Tips and Requirements

Make good use of **your time in class**

Listening

Thinking

Taking notes

Expend **your free time**

Preview and Review

Programing

Take **a pen and some paper** with you

Notes

Exercises

What Data Structures Is About?

All programs **manipulate data**.

So, all programs **represent data** in some way.

Data manipulation requires an **algorithm**.

So, we shall study **ways to represent data and algorithms to manipulate these representations**.

Data structures is concerned with the **representation and manipulation** of data.

In computer science, a **data structure** is a particular way of **storing and organizing data** in a computer so that it can be **used efficiently**.

The study of **data structures** is **fundamental** to Computer Science & Engineering.

Chapter 1

Basic Concepts

- In this chapter, we will introduce and discuss necessary methodologies and techniques for design and implement **large-scale software systems**:
 - ✓ **Data abstraction and encapsulation**
 - ✓ **Algorithm specification**
 - ✓ **Performance analysis and measurement**
 - ✓ **Recursive programming**

1.1 Overview: System Life Cycle

Good programmers regard **large-scale computer programs a system.**

As systems, these programs undergo a development process called the **system life cycle:**

(1) Requirements

(2) Analysis

(3) Design

(4) Refinement and coding

(5) Verification

(1) Requirements

All large programming projects begin with a set of specifications that define the **purpose of the project**.

- ✓ describe the input and output

(2) Analysis

In this phase, we begin to break the problem into **manageable pieces**. Two approaches to analysis:

- ✓ bottom-up analysis (older, **unstructured strategy** that places **an early emphasis on the coding fine points**)
- ✓ top-down analysis (begins by **developing a high-level plan for dividing the program into manageable segments**, preferred)

(3) Design

The designer approaches the system from the perspectives of **the data objects** that the program needs and **the operations** performed on them

- ✓ The first perspective leads to: the **creation of abstract data types**
- ✓ The second perspective requires: **the specification of algorithms and a consideration of algorithm design strategies.**

Example: designing a teaching scheduling system of a university

Typical data objects: student, courses, teachers...

Typical operations: inserting, removing, searching...

(4) Refinement and coding

- ✓ choose **representations** for data objects
- ✓ **write algorithms** for each operation on them

(5) Verification

- ✓ correctness proofs
- ✓ testing with a variety of input data
- ✓ removing errors

1.3 Data Abstraction and Encapsulation

(p.7)Data Encapsulation (or information Hiding) is the concealing of the implementation details of a data object from the outside world.

(p.7)Data Abstraction is the separation between the specification of a data object and its implementation.

The DVD player example:

(1) All our interactions with a DVD player are made through **buttons** on the control panel (or remote control) such as **PLAY**, **STOP** and **PAUSE**. **The DVD player is packaged so that we cannot directly interact with the circuitry inside. So the *internal representation* of the DVD player is *hidden* from the user.** This is the principle of *encapsulation*.

(2) The instruction manual of the DVD player tells us what the DVD player is supposed to do if we press a particular button (i.e. the PLAY button). It does not tell us *how* this function is implemented inside the DVD player. So **the instruction manual makes a clear distinction between *what* the DVD player does and *how* it does it.**

This distinction is the principle of abstraction.

(p.8)A Data Type is a collection of *objects* and a set of *operations* that act on those objects.

Whether your program is dealing with predefined and user-defined (char, int, arrays, structs, classes), these two aspects must be considered: **objects and operations**.

(p.8)An Abstract Data Type (ADT) is a data type that is organized in such a way that the **specification of the objects and the specification of the operations on the objects is separated from the representation of the objects and the implementation of the operations**.

ADT1.1: Abstract data type NaturalNumber

ADT NaturalNumber is

objects: An ordered subrange of the integers starting at zero and ending at the maximum integer (MAXINT) on the computer.

functions:

for all $x, y \in \text{NaturalNumber}$; $\text{TRUE}, \text{FALSE} \in \text{Boolean}$ and
where $+, -, <, ==$, and $=$ are the usual integer operations

$$Zero():NaturalNumber ::= 0$$

```
IsZero(x):Boolean ::= if (x==0) IsZero=true
                        else IsZero=false
```

```

Add(x,y):NaturalNubmer ::=if(x+y<=MAXINT)Add=x+y
                           else Add=MAXINT

```

```
Equal(x,y):Boolean ::= if(x==y) Equal=True
                        else Equal=FALSE
```

Successor(x):NaturalNumber ::= if($x == \text{MAXINT}$) Successor = x
else Successor = $x + 1$

Subtract(x,y):NaturalNumber ::= if($x < y$) Subtract = 0
else Subtract = $x - y$

Benefits from data abstraction and data encapsulation

(1) Simplification of software development:

The chief advantage of data abstraction is that it facilitate the decomposition of the complex task of developing a software system into a number of simpler subtasks.

□ Consider the following situation:

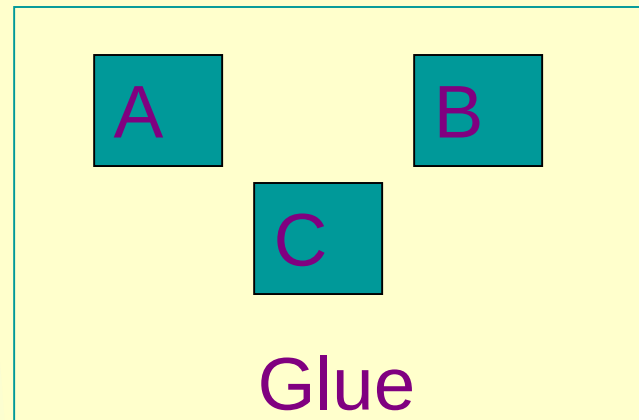
- ✓ A problem has been assigned to (a) a team of programmers or to (b) a single programmer.
- ✓ Suppose that a **top-down review** of the problem results in a decision that: **three data types A, B, C** will be used along with some additional code (**Glue**) to facilitate interactions among the three data types.

- ✓ (a) **Scenario 1-a team of 4 programmers**: one for A, one for B, one for C, one for Glue

No programmer needs to know **how** the other programmers implement their portion of the code.

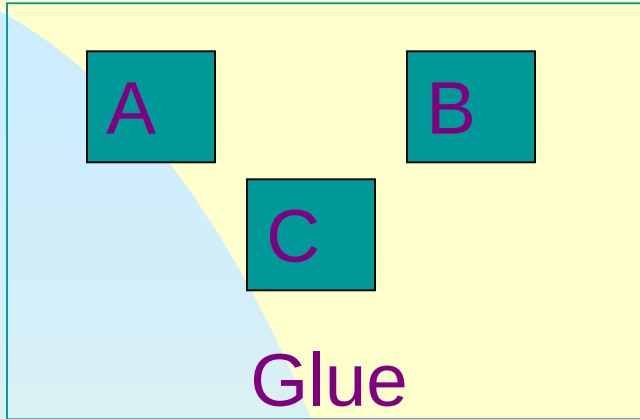
- ✓ (b) **Scenario 2-a single programmer**: implement A, B, C and Glue one by one.

Data abstraction helps reduce the number of things the programmer has to keep in mind at any time.



(2) Simplifying testing and debugging:

Code with **data abstraction**



(a)

Code **without data abstraction**



(b)

In (a): A, B, C can be **tested and debugged separately**. Then the entire program can be tested. If a bug is detected at this stage, it is unlikely that there is a bug in A, B and C, so the bug will most likely be in the glue represented by the unshaded portions of (a).

In (b): **data abstraction is not used**, the amount of code to be searched to find the bug is longer (unshaded areas of (b)).

(3) Reusability:

Data abstraction and encapsulation typically give rise to data structures that are **implemented as distinct entities** of a software system. This makes it easier to extract the code for a data structure and its operations from a software system and use it in another system.

(4) Benefits from modifications to the representation of a data type:

A change in the internal implementation of a data type will not affect the rest of the program as long as its interface does not change.

1.5 Algorithm Specification

An **algorithm** is a **finite set of instructions** that, if followed, accomplishes a particular task.

All algorithms must satisfy the following criteria:

- (1) **Input** Zero or more quantities externally supplied.
- (2) **Output** At least one quantity is produced.
- (3) **Definiteness** Each instruction is clear and unambiguous.
- (4) **Finiteness** Terminates after a finite number of steps for all cases.
- (5) **Effectiveness** Each instruction must be basic enough to be carried out and feasible

Compare: algorithms and programs

---Finiteness

How to describe an algorithm:

- **Using natural language** with the resulting instructions definite.
- **Graphic representations** called flowcharts for small and simple algorithm.
- In this text, we use C++ or the combination of English and C++.

Two examples should help to illustrate the process of **translating a problem into an algorithm**

Problem1: Suppose we must devise a program that sorts a collection of $n \geq 1$ integers.

A simple solution: From those integers that are currently unsorted, find (select) the smallest and place it next in the sorted list.

But it is not an algorithm: it does not tell us where and how the integers are initially stored or where we should place the result.

So we assume that the integers are stored in **an array**, a , such that the i th integer is stored in $a[i-1]$. We get the algorithm described partially in C++ and partially in English:

Program 1.7: Selection sort algorithm

//written partially in C++ and partially in English

```
for (int i=0; i<n; i++){  
    examine a[i] to a[n-1] and suppose the smallest integer  
    is at a[j];  
    interchange a[i] and a[j];  
}
```

To turn Program 1.7 into **a real C++ program**, two clearly defined subtasks remain: **finding the smallest integer and interchanging it with a[i].**

- **We can solve the latter problem by using the:**

```
temp=a[i]; a[i]=a[j]; a[j]=temp;
```

We implement this by calling the standard C++ function `swap(a[i], a[j])`, whose arguments are passed by reference.

- **The first subtask to finding the smallest integer, we can solved by assuming the minimum is `a[i]`, checking `a[i]` with `a[i+1]`, `a[i+2]`, ..., and whenever a smaller element is found, regarding it as the new minimum. Eventually `a[n-1]` is compared to the current minimum, and we are done.**
- **At last, we get the function `SelectionSort(Program 1.8)`**

Program 1.8: Selection sort

```
void SelectionSort (int *a, const int n)
```

```
{//Sort the n integers a[0] to a[n-1] into nondecreasing order.
```

```
  for (int i=0; i<n; i++)
```

```
  {
```

```
    int j=i;
```

```
    //find smallest integer in a[i] to a[n-1]
```

```
    for (int k=i+1; k<n; k++)
```

```
      if(a[k]<a[j])j=k;
```

```
    swap(a[i] and a[j]);
```

```
  }
```

Problem2: Assume that we have $n \geq 1$ distinct integers that are **already sorted** and stored in the array $a[0]$, $a[1], \dots, a[n-1]$. Our task is to determine if the integer x is present and if so to return j such that $x = a[j]$; otherwise return -1;

Solution method:

Let $left$ and $right$ denote the left and right ends of the list to be searched, respectively. Initially, $left = 0$ and $right = n - 1$. Let $middle = (left + right) / 2$ be the middle position in the list. If we compare $a[middle]$ with x , we obtain one of three results:

(1) $x < a[\text{middle}]$. In this case, if x is present, it must be in the positions between 0 and $\text{middle}-1$. Therefore, we set right to $\text{middle}-1$;

(2) $x == a[\text{middle}]$. In this case, we return middle ;

(3) $x > a[\text{middle}]$. In this case, if x is present, it must be in the positions between $\text{middle}+1$ and $n-1$. So, we set left to $\text{middle}+1$;

If x has not been found and there are still integers to check, we recalculate middle and continue to search.

So we get a pseudo-C++ function:

Program 1.9: Algorithm for binary search

//Assume that $n \geq 1$ distinct integers are already sorted and stored in the array $a[0] \dots a[n-1]$.

```
int BinarySearch (int *a, const int x, const int n)
{ //Search the sorted array  $a[0] \dots a[n-1]$  for x
  Initialize left and right;
  while (there are more elements)
  { let middle be the middle element;
    if ( $x < a[middle]$ ) set right to middle-1;
    else if ( $x > a[middle]$ ) set left to middle + 1;
    else return middle;  }
  Not found;
}
```

It is easy to be turn Program 1.9 into a real C++ function.

Program 1.10: C++ function for binary search

```
int BinarySearch (int *a, const int x, const int n)
{ //Search the sorted array a[0]...a[n-1]for x
    int left=0, right=n-1;
    while(left<=right)
    { //there are more elements
        int middle=(left+right)/2;
        if(x<a[middle]) right = middle-1;
        else if(x>a[middle]) left = middle +1;
        else return middle;
    } //end of while
    return -1; //not found;
}
```

Recursive Algorithms

A *recursive function* is a function that calls itself.

- ✓ **Direct recursion:** A function call itself before it is done;
- ✓ **Indirect recursion:** A function call other functions that again invoke the calling function.

Similar to **mathematical induction**

iteration vs recursion

- Anything that can be solved *iteratively* can be solved *recursively* and **vice versa**.
- Sometimes a recursive solution can be expressed more *simply* and *succinctly* than an iterative one.

factorial Function ($n!$)

$$\text{factorial}(0) = 1 \text{ (by definition)} = 1$$

$$\text{factorial}(1) = 1*1 = 1*\text{factorial}(0)$$

$$\text{factorial}(2) = 2*1 = 2*\text{factorial}(1)$$

$$\text{factorial}(3) = 3*2*1 = 3*\text{factorial}(2)$$

$$\text{factorial}(4) = 4*3*2*1 = 4*\text{factorial}(3)$$

$$\text{factorial}(5) = 5*4*3*2*1 = 5*\text{factorial}(4)$$

$$\text{factorial}(6) = 6*5*4*3*2*1 = 6*\text{factorial}(5)$$

Recursive Definition of *factorial*(*n*)

$$\begin{aligned} \text{factorial}(n) = & \quad 1 && \text{if } n = 0 \\ & n * \text{factorial}(n-1) && \text{if } n > 0 \end{aligned}$$

- How would we implement this in C++ ?

Function Definition:	C++ Implementation:
$ \begin{aligned} &factorial(n) = \\ &1 \qquad \qquad \qquad \text{if } n = 0 \\ &n * factorial(n-1) \quad \text{if } n > 0 \end{aligned} $	<pre> int factorial(n) { if(n == 0) return 1; else return n*factorial(n-1); } </pre>

Understanding Recursion

- You can think of a **recursive function call** as if it were calling a completely **separate function**.
- In fact, the ***operations*** that can be performed by both functions is **the same**, but the ***data*** input to each is **different**.

Understanding Recursion (Cont'd.)

```
int factorialA( int    n )  
{  
  if( n == 0 ) return 1;  
  else  
    return  
      n*factorialB( n-1 );  
}
```

```
int factorialB( int    m )  
{  
  if( m == 0 ) return 1;  
  else  
    return  
      m*factorialC(m-1);  
}
```

- If factorialB() and factorialC() perform **the same operations** as factorialA(), then factorialA() can be used in place of them.

Example: *factorial(3)*

`factorial(3)`: $n = 3$ calls `factorial(2)`

`factorial(2)`: $n = 2$ calls `factorial(1)`

`factorial(1)`: $n = 1$ calls `factorial(0)`

`factorial(0)`: returns 1 to `factorial(1)`

`factorial(1)`: $1 * \text{factorial}(0)$ becomes $1 * 1 = 1$

: returns 1 to `factorial(2)`

`factorial(2)`: $2 * \text{factorial}(1)$ becomes $2 * 1 = 2$

: returns 2 to `factorial(3)`

`factorial(3)`: $3 * \text{factorial}(2)$ becomes $3 * 2 = 6$

: returns 6

Questions for Constructing Recursive Solutions

- **Strategy:** Can you define the original problem in terms of smaller problem(s) of the same type?
 - Example: $factorial(n) = n * factorial(n-1)$ for $n > 0$
- **Progress:** Does each recursive call diminish the size of the problem?
- **Termination:** As the problem size diminishes, will you eventually reach a “*base case*” that has an easy (or trivial) solution?
 - Example: $factorial(0) = 1$

X^n Function

$$X^n = 1 \quad \text{if } n = 0 \text{ (base case)}$$

$$X^n = X * X^{(n-1)} \quad \text{if } n > 0$$

This can easily be translated into C++. However, a *more efficient* definition is possible:

$$X^n = 1 \quad \text{if } n = 0 \text{ (base case)}$$

$$X^n = [X^{(n/2)}]^2 \quad \text{if } n > 0 \text{ and even}$$

$$X^n = X * [X^{(n-1)/2}]^2$$

C++ Implementation of X^n

```
double power( double X, int n )
{
    double HalfPower;
    if( n == 0 ) return 1;
    if( n % 2 == 0 ) // n is even
    {
        Halfpower = power( X, n/2 );
        return HalfPower*HalfPower;
    }
    Halfpower = power( X, (n-1)/2 ); // n is odd
    return X*HalfPower*HalfPower;
}
```

Fibonacci Sequence

The first two terms of the sequence are 1, and **each succeeding term is the sum of the previous pair.**

1

1

$$1 + 1 = 2$$

$$1 + 2 = 3$$

$$2 + 3 = 5$$

$$3 + 5 = 8$$

$$5 + 8 = 13 \dots, \text{ or}$$

1	1	2	3	5	8	13	21	34
55	89	144	233	377	610	...		

Fibonacci Sequence (Cont'd.)

Function Definition:	C++ Implementation:
$\text{fib}(1) = 1$ (base case) $\text{fib}(2) = 1$ (base case) $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, for $n > 2$	<pre>int fib(int n) { if(n <= 2) return 1; else return fib(n-1) + fib(n-2); }</pre>

- 2 base cases and 2 simpler problems of the “same kind”

- **Very inefficient:** $\text{fib}(7)$ will call $\text{fib}(3)$ five times!

Mad Scientist's Problem

A mad scientist wants to make a straight chain of length n out of pieces of lead and plutonium. He knows that if he puts two pieces of plutonium next to each other, the whole chain will explode. How many safe, linear chains are there?

Example: $n = 3$

L L L (safe)

L L P (safe)

L P L (safe)

L P P (unsafe)

P L L (safe)

P L P (safe)

P P L (unsafe)

P P P (unsafe)

Result: 5 safe chains

Mad Scientist (Cont'd.)

Let:

$C(n)$ = number of safe chains of length n

$L(n)$ = number of safe chains of length n *ending with lead*

$P(n)$ = number of chains of length n *ending with plutonium*

Now, the total number of safe chains of length n must be the sum of those that end with lead and those that end with plutonium, namely

$$C(n) = L(n) + P(n)$$

Mad Scientist (Cont'd.)

Note that we make a chain of length n by adding to a chain of length $n-1$.

So, consider all chains of length $n-1$. Note that we can add a piece of lead to the end of each of these, since this will not make the chain unsafe.

Therefore,

$$L(n) = C(n-1)$$

Mad Scientist (Cont'd.)

Consider again all chains of length $n-1$. Note that we can add a piece of plutonium to the end of only the chains that end with lead.

Therefore,

$$P(n) = L(n-1)$$

Mad Scientist (Cont'd.)

Substituting formulas for $L(n)$ and $P(n)$ in the formula for $C(n)$ we see that

$$\begin{aligned}C(n) &= L(n) + P(n) \\&= C(n-1) + L(n-1) \\&= C(n-1) + C(n-2),\end{aligned}$$

Note that this is the Fibonacci recursion!

However, the base case(s) are different from the original Fibonacci:

$$C(1) = 2 \quad L \text{ or } P$$

$$C(2) = 3 \quad LL \text{ or } LP \text{ or } PL$$

Binary Search: *Telephone Book*

- **Problem:** Search the telephone book for someone's phone number.
- **Binary Search Strategy:**
 - a) Open the book somewhere near **the middle**.
 - b) If the the person's name is **in the first half**, ignore the second half, and search the first half, starting again at step a).
 - c) If the the person's name is **in the second half**, ignore the first half, and search the second half, starting again at step a).
 - d) If the person's name is **on a given page**, scan the **page for the person's name**, and find the phone number associated with it.

Binary Search: *Search an Array*

- **Problem:** Given an array, $A[]$, of n integers, sorted from smallest to largest, determine whether value v is in the array.
- **Binary Search Strategy:**
 - If $n = 1$ then check whether $A[0] = v$. Done.
 - Otherwise,
 - ✓ find the **midpoint** of $A[]$.
 - ✓ If $v > A[\text{midpoint}]$ then recursively search the second half of $A[]$.
 - ✓ If $v \leq A[\text{midpoint}]$ then recursively search the first half of $A[]$.

Search an Array: C++ Implementation

```
int binarySearch( int A[ ], int v, int first, int last )
{
    if( first > last ) return -1;    // v not found in A[ ]

    int mid = (first + last)/2;    // set mid to midpoint

    if( v == A[mid] ) return mid;
    if( v < A[mid] ) return binarySearch( A, v, first, mid-1 );
    return binarySearch( A, v, mid+1, last );
}
```

1.7 Performance Analysis and Measurement

(p.38)Definitions:

The **Space complexity** of a program is the amount of **memory** it needs to run to completion.

The **Time complexity** of a program is the amount of **computer time** it needs to run to completion.

Performance evaluation can be loosely divided into two major phases:

(1) Priori estimates --- **Performance analysis**

(2) Posteriori testing--- **Performance measurement**

Space complexity

The space requirement of program P:

$$S(P) = c + S_p(\text{instance characteristics})$$

- **c is the fixed part that is independent of the characteristics (e.g., number, size) of the inputs and outputs. It typically includes the instruction space, space for simple variables and fixed-size component variables, space for constants.**

We concentrate solely on S_p .

Example 1.10 (p40)

```
float Rsum (float *a, const int n) //compute  $\sum_{i=0}^{n-1} a[i]$  recursively
{
    if (n <=0) return 0;
    else return (Rsum(a,n-1)+a[n-1]);
}
```

- The instances are characterized by n .
- The recursion **stack space** includes space for the formal parameters, the local variables, and the return address.
- Each call to Rsum requires at least 4 words (n , a , **return value**, **return address**)
- Since the depth of recursion is $n+1$, the recursion stack space needed $S_{\text{rsum}}(n) = 4(n+1)$.

Time complexity

The time of a program P:

$$T(P) = c + t_p(\text{instance characteristics})$$

- c is the compile time, the run time is denoted by t_p .
- Because many factors t_p depends on are not known at the time a program is conceived, it is reasonable to **attempt only to estimate t_p** .
- It is not easy to obtain the true value of t_p .
- We count only the number of **program steps**.

Program step

A **program step** is **loosely defined** as a syntactically or semantically **meaningful segment** of a program that has an execution time that is **independent of instance characteristics**.

The number of steps any program statement is to be assigned depends on the nature of that statement.

In P41-43 of the textbook, an detailed assignment of step counts to statements in C++ is discussed.

Our main concern:

how many steps are needed by a program to solve a particular problem instance?

2 ways:

(1) introducing a global new variable, **count**

(2) building a **table**

Example 1.11(P43-44)

```
float Sum (float *a, const int n)
{ //Iterative function for sum
    float s=0;
    count++; //count is global
    for(int i=0;i<n;i++) {
        count++; // for for
        s+=a[i];
        count++; //for assignment
    }
    count++; // for the last time of for
    count++; //for return
    return s;
}
```

It is easy to see that in the for loop, the value of **count** will increase by a total of $2n$. If count is zero to start with, then it will be **$2n+3$** on termination.

Notice the for loop:

- $i=0, i<n$ (✓): 1 step
- $i<n, i++$ (✓): $n-1$ steps
- $i<n$ (✗): 1 step
- total: $n+1$ steps

Example 1.12(P44-45)

```
count=0;
float Rsum (float *a, const int n)
{
    count++; // for if
    if (n <=0) {
        count++; // for return
        return 0;
    }
    else {
        count++; // for return
        return (Rsum(a,n-1)+a[n-1]);
    }
}
```

When analyzing a recursive program for its step count, we often obtain a recursive formula for the step count.

Let $t_{\text{Rsum}}(n)$ be the increase in the value of **count** when the program terminates.

$$\begin{aligned} t_{\text{Rsum}}(0) &= 2, \\ t_{\text{Rsum}}(n) &= 2 + t_{\text{Rsum}}(n-1) \\ &= 2 + 2 + t_{\text{Rsum}}(n-2) \\ &\quad \cdot \\ &\quad \cdot \\ &= 2n + t_{\text{Rsum}}(0) \\ &= 2n + 2 \end{aligned}$$

Example 1.14 Fibonnaci numbers (P49)

```
1 void Fibonnaci (int n) //Program 1.25
2 { // compute the Fibonnaci number  $F_n$ 
3   if (n <=1) cout << n<< endl; { //  $F_0=0$  and  $F_1 =1$ 
4     else { // compute  $F_n$ 
5       int fn; int fnm2=0; int fnm1=1;
6       for (int i=2; i<=n; i++)
7         {
8           fn=fnm1+fnm2;
9           fnm2=fnm1;
10          fnm1=fn;
11        } //end of for
12      cout <<fn<<endl;
13    } //end of else
14 }
```

Notice the for loop:

- $i=2, i \leq n$ (✓): 1 s/e
- $i \leq n, i++$ (✓): $n-2$ s/e
- $i \leq n$ (✗): 1 s/e
- total: n steps

Let us use a **table** to count its total steps.

The **s/e** of a statement is the amount by which **count** changes as a result of the **execution of that statement**.

Line	s/e	frequency	total steps
1	0	1	0
2	0	1	0
3	1 ($n > 1$)	1	1
4	0	1	0
5	2	1	2
6	1	n	n
7	0	$n-1$	0
8	1	$n-1$	$n-1$
9	1	$n-1$	$n-1$

10	1	$n-1$	$n-1$
11	0	$n-1$	0
12	1	1	1
13	0	1	0
14	0	1	0

So

- **for $n > 1$, $t_{\text{Fibonacci}}(n) = 4n + 1$**
- **for $n = 0$ or 1 , $t_{\text{Fibonacci}}(n) = 2$**

Summary of time complexity

- ✓ Generally, the time complexity of a program is given by the **number of program steps**.
- ✓ The number of steps is itself a function of **instance characteristics**.
- ✓ Although **any specific instance may have several characteristics** (e.g. the number of inputs, the number of outputs, the magnitude of the inputs and outputs), **the number of steps is computed as a function of some subset of these**. Usually, we choose those characteristics that are important to us.
- ✓ Sometime, the time complexities are **nice functions of simple instance characteristics**.

- ✓ But, for many programs, the time complexity is **not dependent solely on easily specified characteristics**.
 - E.g. We consider *BinarySearch to find x in $a[0...n-1]$* , and we would like to know how the computing time changes as we change the number of elements n . **But, parameter n is inadequate**. For the same n , the step count varies with the position of x in a .
 - Hence, we define three kinds of **step counts**:
 - **Best-case step count**: The minimum number of steps that can be executed for the given parameters.
 - **Worst-case step count**: The maximum number of steps that can be executed for the given parameters.
 - **Average-case step count**: The average number of steps that can be executed for the given parameters.

Asymptotic Notation (O , Ω , Θ)

- ✓ Our motivation to determine step counts is to be able to **compare the time complexities** of two program that computes the same function and also to **predict the growth in run time** as the instance characteristics change.
- ✓ But determining the **exact step count** (either worst-case or average) of a program **can prove to be an exceedingly difficult task**.
- ✓ Expending much effort to determine the step count exactly is **not very worthwhile**, since the notion of **a step is itself inexact**.

- ✓ Thus, we introduce some terminology that will enable us to make **meaningful (but inexact) statements** about the **time and space complexities** of a program.

Asymptotic Notation (O)

Definition [O]: $f(n)=O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq c g(n)$ for all $n, n > n_0$.

Example 1.15:

$$3n+2=O(n),$$

$$6 \cdot 2^n + n^2 = O(2^n), \dots,$$

$$10n^2 + 4n + 2 \neq O(n)$$

- ✓ Note $g(n)$ is an **upper bound**. It does **not** say anything about **how good** this bound is.
- ✓ For example, $n=O(n^2)$, $n=O(2^n)$, ..., **are all correct**.
- ✓ For the statement $f(n)=O(g(n))$ to be informative, $g(n)$ should be **as small as possible**.
- ✓ So, we shall often say $3n+3=O(n)$, we shall almost never say $3n+3 = O(n^2)$.

Theory 1.2: if $f(n)=a_m n^m + \dots + a_1 n + a_0$, then $f(n)=O(n^m)$.

Asymptotic Notation (Ω)

Definition [Ω]: $f(n) = \Omega(g(n))$ **iff** there exist positive constants c and n_0 such that $f(n) \geq c g(n)$ for all n , $n > n_0$.

Example 1.16:

$$3n+2 = \Omega(n),$$

$$6 \cdot 2^n + n^2 = \Omega(2^n), \dots$$

- ✓ Note $g(n)$ is just an **lower bound**.
- ✓ For example, $n^2 = \Omega(n)$, $2^n = \Omega(n)$, ..., are all correct.
- ✓ For $f(n) = \Omega(g(n))$ to be informative, $g(n)$ should be as large as possible.
- ✓ So we shall say that $3n+3 = \Omega(n)$ and $6 \cdot 2^n + n^2 = \Omega(2^n)$, we shall never say $3n+3 = \Omega(1)$ and $6 \cdot 2^n + n^2 = \Omega(n)$.

Theory 1.3: if $f(n) = a_m n^m + \dots + a_1 n + a_0$, and $a_m > 0$ then $f(n) = \Omega(n^m)$.

Asymptotic Notation (Θ)

Definition [Θ]: $f(n) = \Theta(g(n))$ iff there exist positive constants c_1 , c_2 and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all n , $n > n_0$.

Example 1.17: $3n+2 = \Theta(n)$, $6 \cdot 2^n + n^2 = \Theta(2^n)$, $3n+2 \neq \Theta(n^2)$

The theta notation is more precise than both the “big oh” and omega notations.

$f(n) = \Theta(g(n))$ iff $g(n)$ is both an upper and lower bound on $f(n)$.

We shall almost never say $3n+3=\Theta(3n)$ or $6*2^n+n^2=\Theta(6*2^n)$,

Theory 1.4: if $f(n)=a_m n^m+\dots+a_1 n+a_0$, and $a_m>0$ then $f(n)=\Theta(n^m)$.

How to determine the asymptotic complexity of a program.

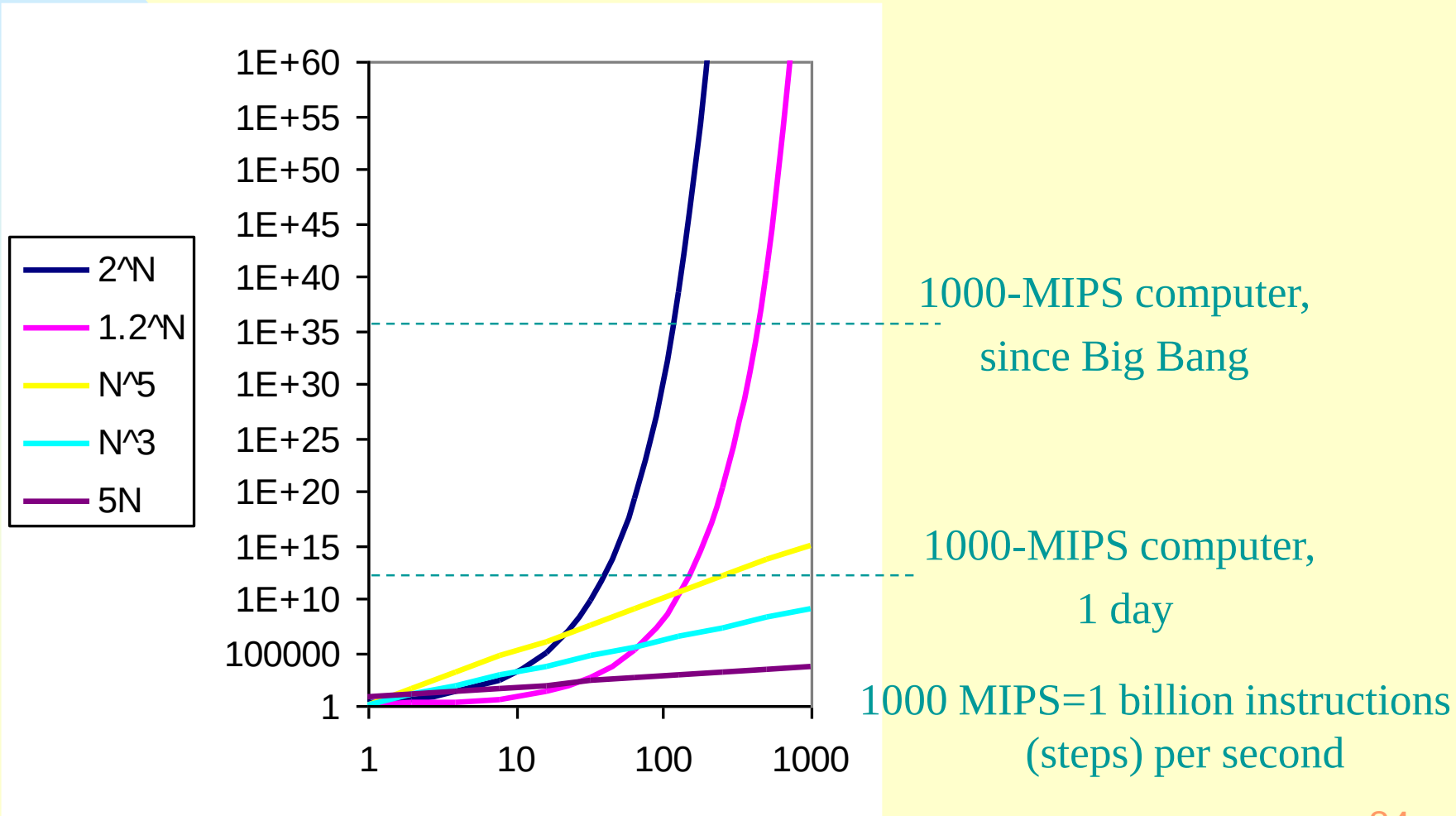
This is usually done by first determining the asymptotic complexity of each statement (or group of statements) in the program and then adding up these complexities.

Common asymptotic complexity

constant:	$O(1)$	
logarithmic:	$O(\log n)$	
linear:	$O(n)$	
log-linear:	$O(n \log n)$	
quadratic:	$O(n^2)$	
polynomial:	$O(n^k)$	(k is a constant)
exponential:	$O(c^n)$	(c is a constant > 1)

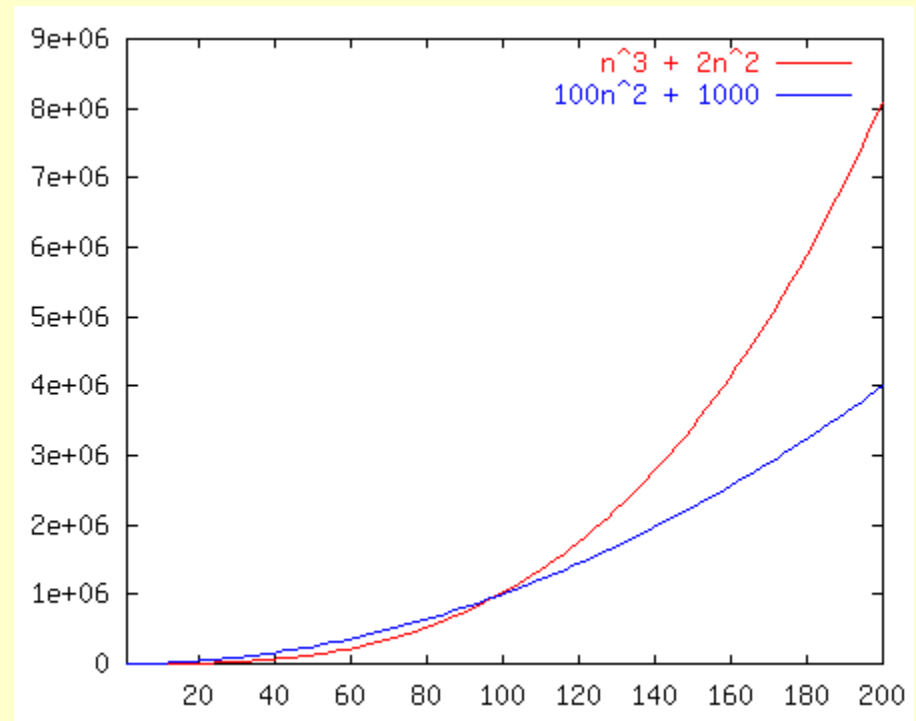
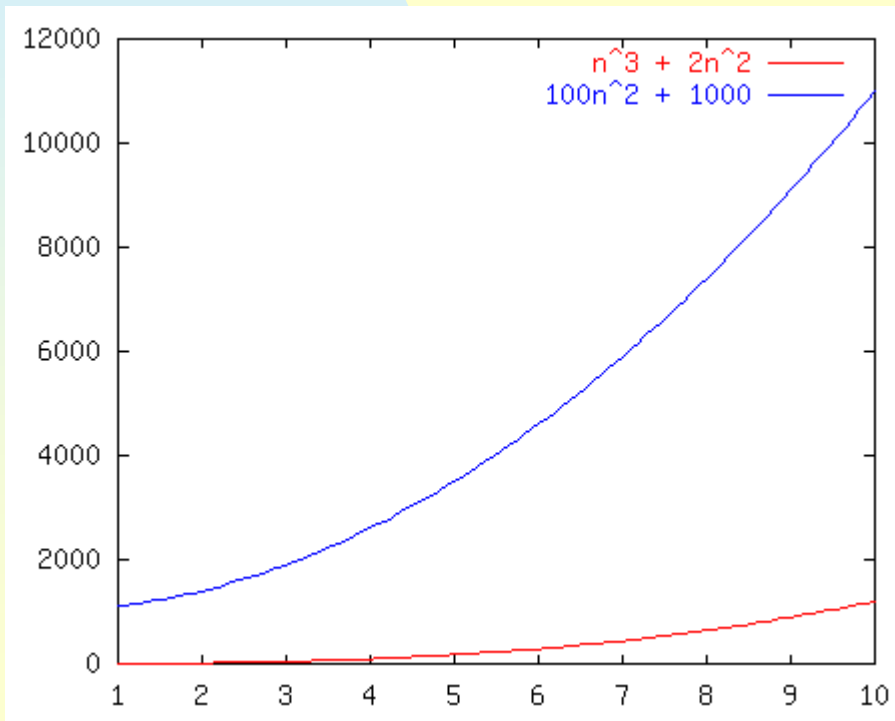
Practical Complexity

How the various functions grow with n ?



Race I

$n^3 + 2n^2$ vs. $100n^2 + 1000$

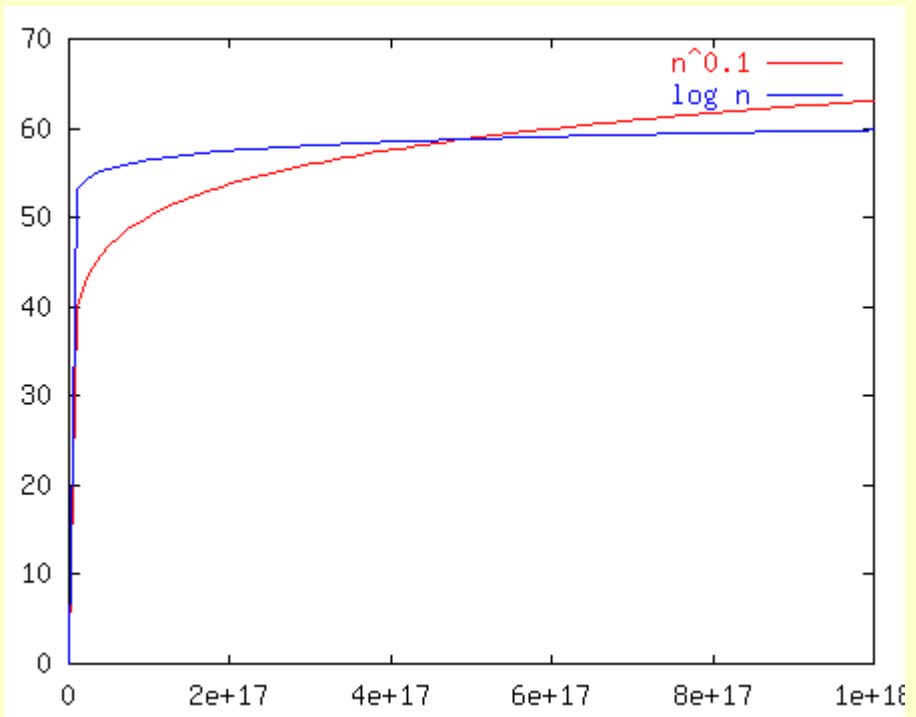
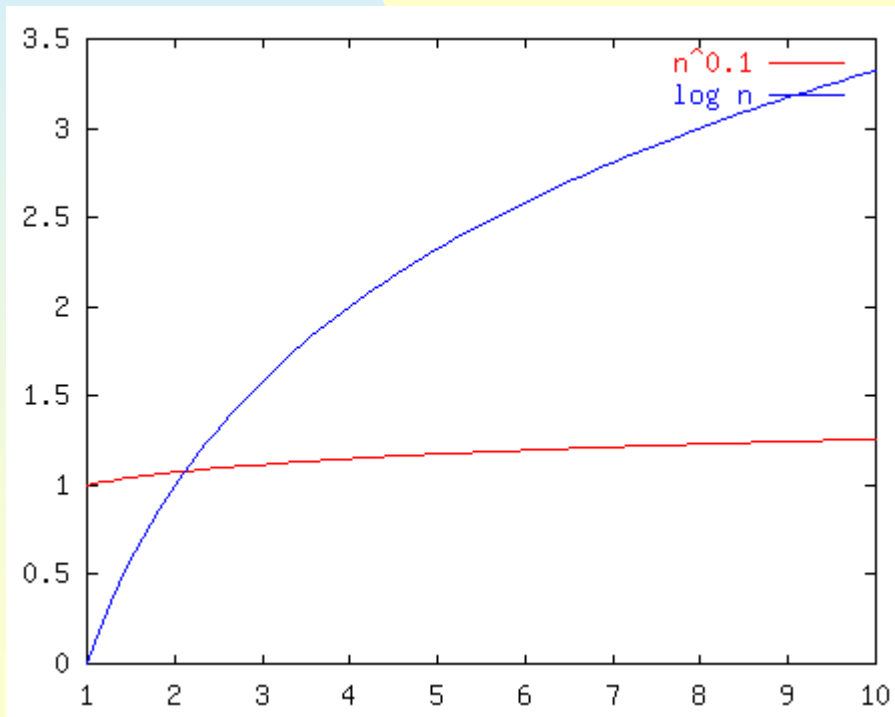


Race II

$n^{0.1}$

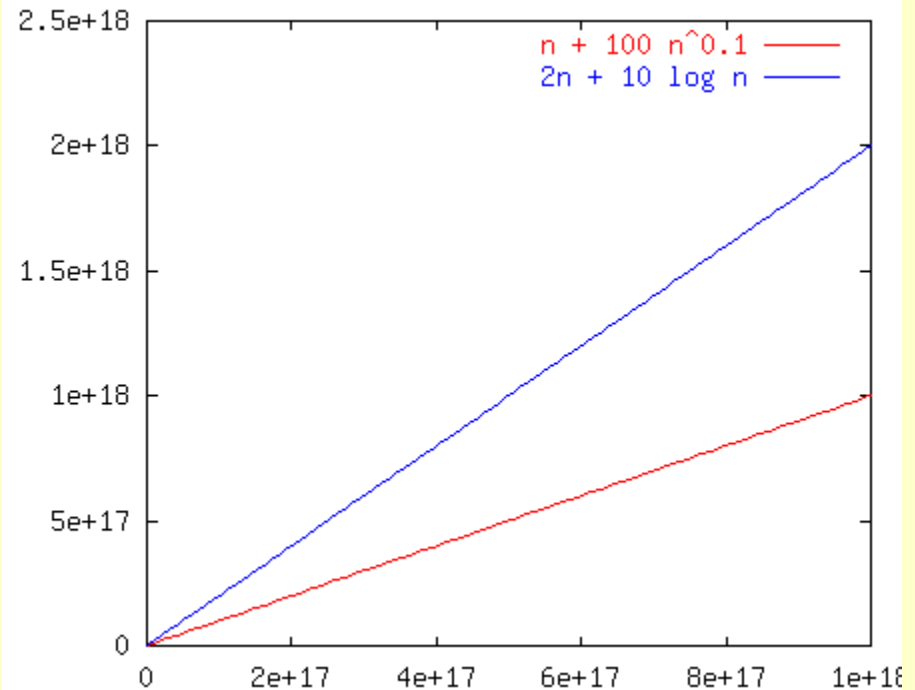
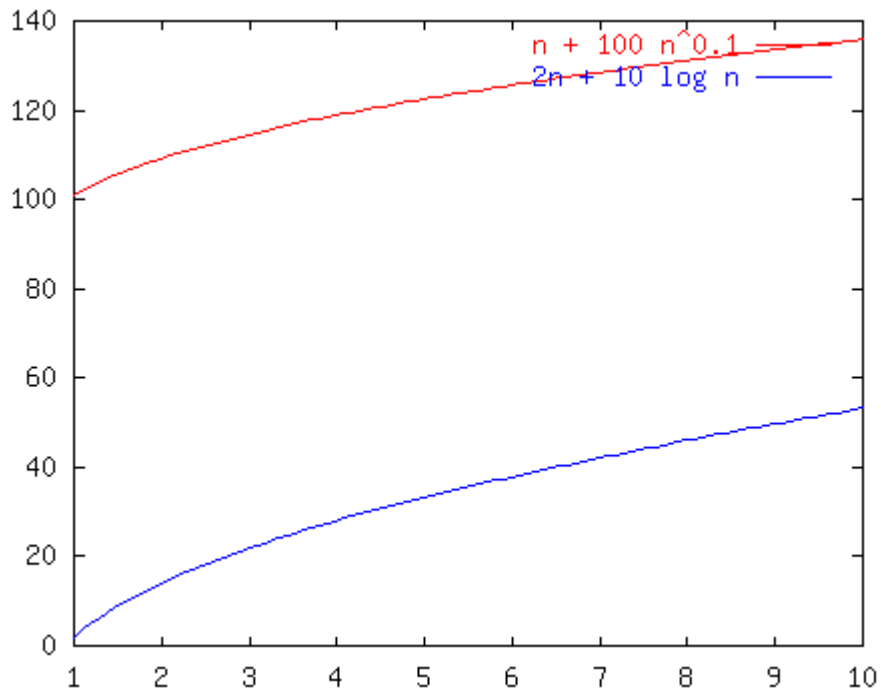
vs.

$\log n$



Race III

$n + 100n^{0.1}$ vs. $2n + 10 \log n$

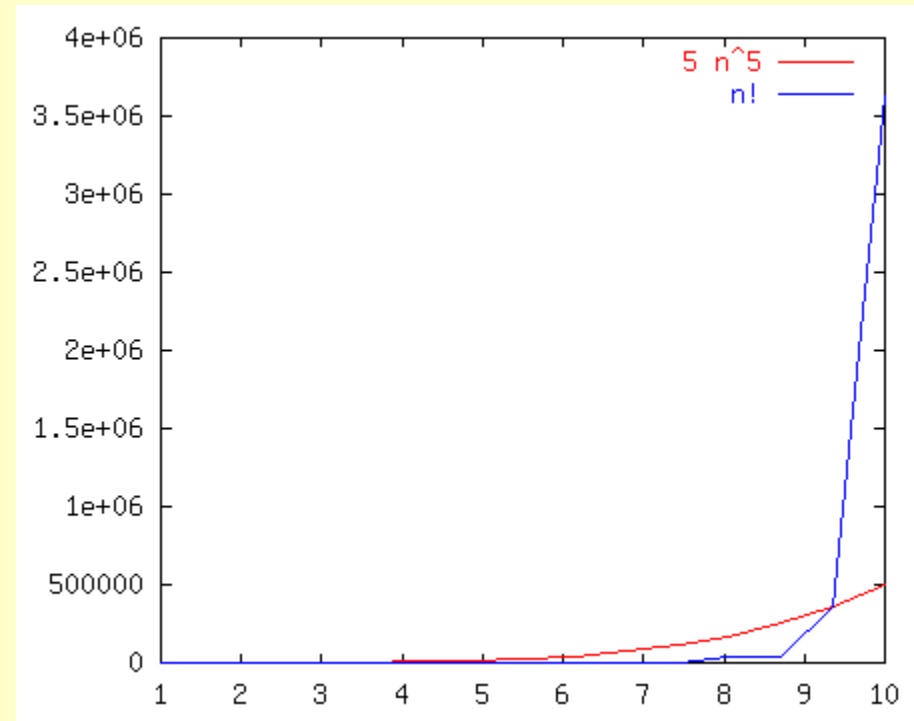
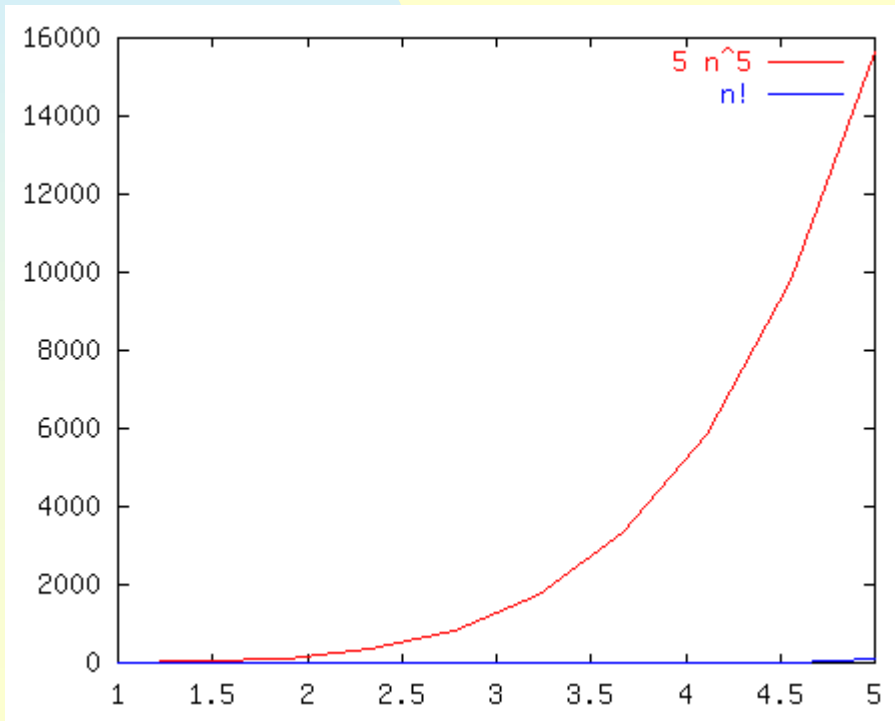


Race IV

$5n^5$

vs.

$n!$



Race V

$$n^{-15} 2^n / 100$$

vs.

$$1000n^{15}$$

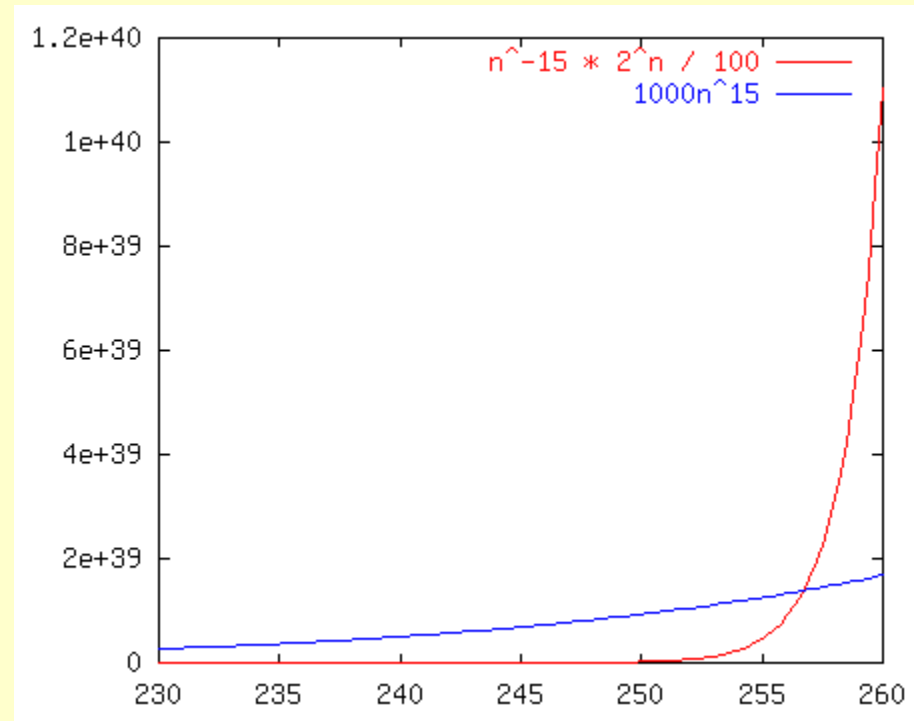
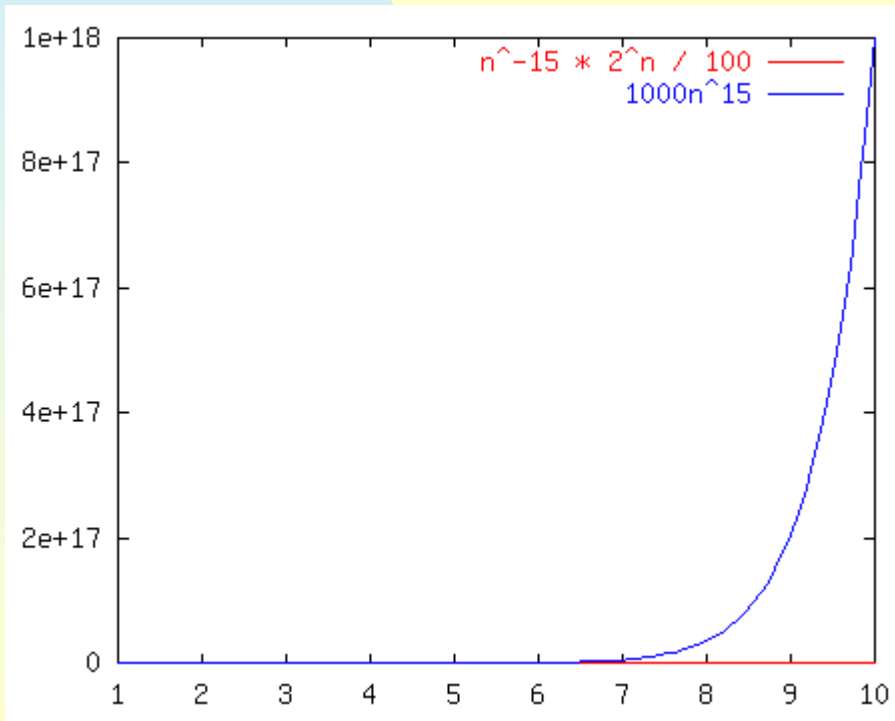


Table 1.8: Times on a 1-billion-steps-per-second (1000 MIPS) computer

n	n	$n \log_2 n$	n^2	n^3	n^4	n^{10}	2^n
10	.01 μ s	.03 μ s	.1 μ s	1 μ s	10 μ s	10s	1 μ s
20	.02 μ s	.09 μ s	.4 μ s	8 μ s	160 μ s	2.84h	1 ms
30	.03 μ s	.15 μ s	.9 μ s	27 μ s	810 μ s	6.83d	1 s
40	.04 μ s	.21 μ s	1.6 μ s	64 μ s	2.56ms	121d	18m
50	.05 μ s	.28 μ s	2.5 μ s	125 μ s	6.25ms	3.1y	13 d
100	.1 μ s	.66 μ s	10 μ s	1ms	100 ms	3171y	4*10 ¹³ y
10 ³	1 μ s	9.66 μ s	1ms	1s	16.67m		
10 ⁴	10 μ s	130 μ s	100ms	16.67m	115.7d		
10 ⁵	100 μ s	1.66ms	10s	11.57d	3171y		

From a practical stand-point, it is evident that for reasonably large n (say $n > 100$), only programs of small complexity (such as $n \log n$, n^2 , n^3) are feasible.

Performance Measurement

- ✓ Performance measurement is concerned with obtaining the **actual space and time requirements** of a program.
- ✓ The time needed for **compilation** is not concerned with.
- ✓ To time a **short event** it is necessary to **repeat** it several times and divide the total time for the event by the number of repetitions.

Example: Measuring the worst-case performance of the **sequence search** function.

Program 1.27: Sequential search

```
int SequentialSearch (int *a, const int n, const int x )  
{ // Search a[0:n-1].  
    int i;  
    for (i=0; i < n && a[i] != x; i++;)  
        if (i == n) return -1;  
    else return i;  
}
```

```

void TimeSearch ( )
{
    int a[1000], n[20];
    const long r[20] = {300000, 300000, 200000, 200000,
        100000, 100000, 100000, 80000, 80000, 50000, 50000,
        25000, 15000, 15000, 10000, 7500, 7000, 6000, 5000,
        5000 }; //set the replication factors for different n

    for ( int j=0; j<1000; j++ ) a[j] = j+1; //initialize a
    for ( j=0; j<10; j++ ) { //set values of n, the max value is 1000
        n[j] = 10*j; n[j+10] = 100*(j+1 );
        //n=0,10,20,...90,100,200,...1000
    }

    cout << “ n      total      runTime” << endl;
}

```

```

for ( j=0; j<20; j++ ) {
    long start, stop;
    time (&start);                                // start timer
    for ( long b=1; b<=r[j]; b++ )
        int k = seqsearch(a, n[j], 0 ); //unsuccessful search
    time (&stop);                                // stop timer
    long totalTime = stop - start;
    float runTime = (float) (totalTime) / (float)(r[j]);
    cout << " " << n[j] << " " << totalTime << " " << runTime
        << endl;
} //end of outer for
}

```

Assume the function **time(hsec)** that returns in the variable *hsec* the current time in hundredths of a second.

The results of running *TimeSearch* are as in the next slide.

n	total	runTime	n	total	runTime
0	241	0.0008	100	527	0.0105
10	533	0.0018	200	505	0.0202
20	582	0.0029	300	451	0.0301
30	736	0.0037	400	593	0.0395
40	467	0.0047	500	494	0.0494
50	565	0.0056	600	439	0.0585
60	659	0.0066	700	484	0.0691
70	604	0.0075	800	467	0.0778
80	681	0.0085	900	434	0.0868
90	472	0.0094	1000	484	0.0968

Figure 1.6: Worst-case run times for Program 1.27

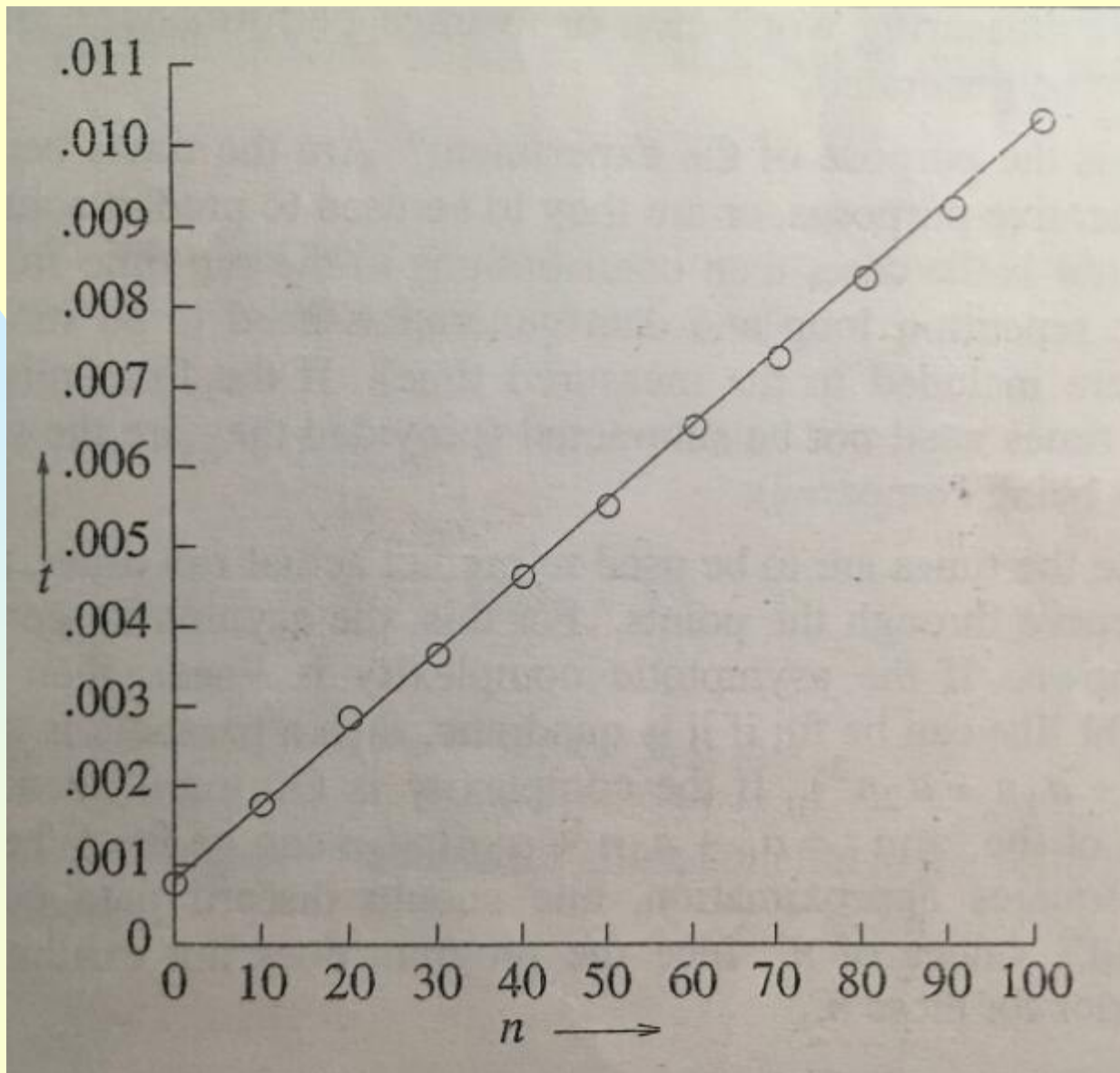


Figure 1.1: Plot of the data in Figure 1.6

To obtain the runtime of a program, we need to plan the experiment. The following issues need to be addressed during the planning stage:

- (1) **Accuracy** of the clock
- (2) **Repetition** factor
- (3) Generating suitable **test data**
- (4) **Purpose**: for comparing or for predicting?
- (5) In case the times are to be used to **predict actual runtimes**, we need **fit a curve** through points. For this, the asymptotic complexity should be known.

Experiment(s) and Exercise(s)

Exercise(s):

- P69:3,4,5,6,7

Experiment(s):

- P69:1,10